

FPS: Flexible Payment System

Adithya Bhat
Visa Research

Srinivasan Raghuraman
Visa Research
Massachusetts Institute of Technology

Panagiotis Chatzigiannis
Visa Research

Duc V. Le
Visa Research

Mohsen Minaei
Visa Research

Abstract—Existing payment systems make fixed trade-offs between performance and security assumptions. Traditional centralized systems like Visa assume synchronous networks and crash faults to achieve high throughput, while blockchain-based systems (e.g., Algorand, Aptos) adopt Byzantine fault tolerance and partial synchrony for stronger security at the cost of performance. This rigid approach forces all users to accept the same security-performance trade-off regardless of their individual trust and threat models.

We present a flexible payment system where clients independently choose assumptions about (i) network timing (bounded or partial synchrony), (ii) corruption (static or adaptive), and (iii) faults (crash or Byzantine), supporting eight assumption combinations simultaneously. Unlike traditional systems requiring consensus, our approach uses a novel flexible variant of consistent broadcast where clients external to the protocol verify delivery through cryptographic proofs, eliminating the need for global ordering. We implemented our system in Rust and demonstrated that clients choosing partially synchronous network and crash assumptions achieve +242.1% higher throughput and +70.4% better latency compared to clients with synchronous network and Byzantine assumptions, confirming that our system enables users to optimize their individual security-performance trade-offs.

1. Introduction

Contemporary payment systems provided by the banks assume crash-fault tolerance, using private ledgers to maintain clients’ account balances [1]. Banks, a centralized entity, own and administer these ledgers, and are responsible for the system’s overall maintenance. Clients interact with the system through the banks’ applications and services to conduct payments. These systems rely on trust assumptions, such as using firewalls to protect clients and compensating them in case of security breaches but these assumptions are neither transparent nor configurable by individual users. Decentralized payment systems pioneered by Bitcoin [2] and others [3], [?] introduced transparent public trust assumptions. These systems primarily utilize consensus protocols [4] in which all users *agree* on the system’s state, enabling public trust assumptions and verification of balances.

Traditional payment systems typically rely on fault-tolerant State Machine Replication (SMR), which ensures all nodes view a consistent state, to ensure correctness

of payments. These protocols rely on static assumptions about system behavior, including the network type (e.g., Δ -bounded synchrony or partial synchrony), fault type (crash or Byzantine), and corruption type (static or adaptive). The system incorporates these assumptions in their design and the expectation is that these assumptions hold for the protocol’s lifetime. Additionally, the deployment context constrains assumptions choices. For instance, decentralized payment systems inherently demand consideration of Byzantine faults as the users do not trust the unknown operators.

Clients would benefit from forming their own perspective of the system based on experience, heuristics, or external knowledge. They may adjust their assumptions by monitoring system conditions, adopting conservative parameters for critical payments and relaxed parameters for non-critical ones, and even preferring certain quorums of servers over others. For instance, a client making a USD \$1M transfer may prefer Byzantine fault tolerance and accept higher latency while a USD \$5 coffee payment could use crash fault-tolerance for instant confirmation. Such flexibility is particularly valuable when different transactions demand different security-performance trade-offs.

This motivates our key question: *Can we design a payment system where clients independently choose their security assumptions?* Ideally, an assumption-agnostic SMR protocol would allow clients to apply their own assumptions about the system. This requires a protocol design that is “open to interpretation” by clients with varying assumptions. Bitcoin provides an early example: it does not specify block finalization, allowing clients to choose their own confirmation depth k based on risk tolerance [2], [5], [6]. This leaves finality “open to interpretation” with failure probability $\text{negl}(k)$.

SMR is a general-purpose specification that we can implement in multiple ways depending on the underlying state machine. A common primitive used to realize SMR is a fault-tolerant atomic broadcast protocol. In a nutshell, an atomic broadcast protocol orders all the payments received by the system. However, recent studies [7], [8], [9] have shown that a payment system need not implement atomic broadcast. The intuition here is to forego Byzantine payer support and instead ensure successful payments only for non-faulty payers and payees. They demonstrate that a weaker distributed computing primitive, known as consistent broadcast [10], suffices. Consistent broadcast ensures

agreement only if the designated sender is non-faulty.

Despite these observations, no existing payment system allows clients to flexibly choose their own assumptions within a single system. While some forms of flexible protocols exist [11], [?], they rely on atomic broadcast or offer limited flexibility. This work bridges this gap by enabling client-side assumption flexibility in payment systems.

Contributions. We present a single payment system that uses a variant of consistent broadcast and supports clients with different network, corruption, and fault assumptions on the servers and the system. We call our system a *Flexible Payment System (FPS)* borrowing the terminology from Malkhi et al. [11]. Concretely, we make the following contributions:

We present FPS, a novel flexible payment system enabling clients to optimize their own performance-security trade-offs. Concretely, clients independently choose from two network, two fault, and two corruption assumptions resulting in a combination of **eight** security assumptions on the system. More specifically, clients choose from:

- **two network assumptions** (bounded synchrony or partial synchrony),
- **two types of faults** (crash or Byzantine),
- **two corruption assumptions** (static or adaptive).

We prove that FPS provides the same security guarantees as traditional payment systems when client assumptions match reality.

We implement our protocol in Rust and evaluate it under realistic network conditions. We demonstrate the trade-off between fault-tolerance, efficiency, and trust. We find that stronger security assumptions (Byzantine fault-tolerance, bounded synchrony) reduce performance, while increased trust assumptions (crash faults, partial synchrony) can improve throughput and latency by +242.1% and +70.4% respectively compared to the most conservative assumptions.

1.1. Solution Overview

We achieve this through several key technical insights:

In Fig. 1, we present a logical overview of FPS—our protocol for a Flexible Payment System. The servers run a server-side protocol that maintains a log, which may be inconsistent across other servers. Our protocol ensures that non-faulty clients with the same assumptions will all have a consistent view assuming that their assumptions hold, i.e., if the actual system behavior matches the client’s chosen assumptions (e.g., network delays respect the chosen bound, actual faults do not exceed the threshold, etc.). As part of this log derivation, clients apply local algorithms that filter a subset of the log such that clients can establish a global “before” and “after” relationship among transactions, thereby realizing a secure payment system.

Payments do not require total ordering. Following the approach by Gupta [12] and others [8], [9], [13], we recognize that implementing a payment system does not require a total order (as implemented by systems that implement Atomic Broadcast [10] for all the payments. Intuitively, this is because of two reasons. First, a payment is an operation

that only the owner of the account can initiate. So, the system needs to ensure that the ordering is on a per-account level and not across accounts. Once we achieve this, we can verify any cross account payment using recursive checks across all intermediate payers. Thus, we can intuitively ensure that each node ‘agrees’ that the state of an “owner” is correct and can convince others of the same.

Hybrid Byzantine Consistent Broadcast with Source Ordering. Rachid et al. [7] show that a *secure broadcast with source order* suffices to implement a payment system. We extend this result to show that a weaker primitive called hybrid Byzantine Consistent Broadcast with Source Order (hybrid BCB with SO) is sufficient to implement the same. The hybrid here refers to the fact that we have two classes of nodes: *servers* and *clients*. Clients initiate broadcasts and process outputs, while servers support the broadcast abstraction. The consistency property in hybrid BCB ensures that there is only one output but does not necessitate totality, i.e., all clients output it. This is strictly weaker than the requirements of a secure broadcast, which requires that the outputs be consistent and that all clients output them. Additionally, we show how to efficiently realize the additional Source Order property from hybrid BCB by incorporating sequence numbers into each payer’s transactions, which might be of independent interest.

Making networks flexible. Under bounded synchronous network assumptions, nodes can infer that an operation is valid based on the absence of complaints from servers. Therefore, to support clients with bounded synchrony assumptions, our server-side protocol incorporates multicasting payments. If a server detects any conflicting payments for a sequence number, it sends an error to the client. As a result, clients with bounded synchrony assumptions can obtain higher fault tolerance and smaller quorum sizes. To support partially synchronous network assumptions, clients generally rely on a majority of non-faulty servers agreeing on the transaction. Since the two network assumptions are non-conflicting, clients making bounded synchrony assumptions use timeouts, and clients making asynchrony assumptions count the acknowledgments, and simultaneously validate payments.

Make faults flexible. We allow clients to choose between crash and Byzantine faults. The type of fault affects the number of acknowledgments needed to finalize a payment. In a *synchronous* network with a crash assumption, clients finalize payments by waiting for an absence of complaints without waiting for any acknowledgments. Under a Byzantine assumption, clients similarly wait for no complaints but security requires that the honest servers are in majority. In *partially asynchronous* networks, clients finalize payments by securing agreement from a majority of non-faulty servers. Under both assumptions, clients wait for acknowledgments from more than half the honest nodes. Thus, clients choose a quorum size based on the assumptions on the network, fault type and fault number. We derive quorum sizes from standard quorum intersection arguments and match the known bounds for consistent broadcast in the respective fault models. Thus, our system allows clients to

freely choose between fault types and adjust the number of acknowledgements it needs to collect accordingly thereby enabling flexibility.

Making corruption flexible. Finally, our protocol allows clients to choose between static and adaptive corruptions. After choosing the network and fault assumption, the clients obtain the quorum sizes. In the static case, clients only need to ensure that the ratio of the bounds need to hold with high probability. Thus, we allow our clients to select a random quorum ensuring that the bounds hold statistically determined by the security parameter κ ensuring that the failure probability is negligible.

2. System Model

We use $\leftarrow$ for assignment, $:=$ for parsing, and $=$ for equality testing.

Our system consists of two types of nodes: *servers* maintaining the payment system and *flexible clients* performing the payments. We assume that the system consists of n servers $\mathcal{P} := \{p_1, \dots, p_n\}$, and any number of clients $\mathcal{C} = \{c_1, \dots\}$, with $\mathcal{N} := \mathcal{P} \cup \mathcal{C}$. We assume that each node has a unique identifier that the other nodes can verify through standard means, such as public key infrastructure (PKI). We assume H to be a collision-resistant hash function.

Our system consists of two node networks: (i) *server-network*: the network between all the servers, and (ii) *client-network*: the network between *flexible clients* and all the servers. We assume that the channels in both networks are authenticated and reliable.

Clocks. We do not assume any global clocks. Instead, we assume that clients use their local clocks for time measurement if needed. We also presume that clients are aware of any skew in their local clocks and that the chosen network parameters and timeouts accommodate these clock skews.

2.1. Network flexibility

A flexible client chooses between two types of networks.

Bounded Synchrony. In this network model, a public delay parameter Δ exists such that if a non-faulty node n_i sends a message m to another node n_j at time T , then n_j receives m by time $T + \Delta$.

We assume *non-lock step synchronous* nodes in this network. The clocks of these nodes may deviate from each other by at most Δ . In practice, we can realize this using a periodic clock synchronization protocol [14], [15].

Partial Synchrony. In this network model [16], the network¹ consists of the following parameters:

- a public delay parameter Δ
- a global stabilization time T_{GST} chosen by the adversary
- a period of synchrony T_L provided by the protocol operating in this network

In any time period of length $T_{\text{GST}} + T_L$, the network functions asynchronously for time period T_{GST} , and is Δ -bounded synchronous for time period T_L .

Note on mixed networks. If a flexible client assumes a bounded-synchronous server-network and a partially synchronous client-network, then we consider the client assumes a partially synchronous network. Furthermore, if a flexible client assumes a partially-synchronous server-network and a bounded-synchronous client-network, we also consider the client assumes a partially synchronous network. We assume that the delay parameter used for partial synchrony is the larger of the delay parameters in both networks.

2.2. Corruption Flexibility

We assume two kinds of corruptions by the adversary. When an adversary $\mathcal{A}$ corrupts a node, it obtains the internal state of the node and takes control of it. For a *static corruption*, clients assume that a PPT adversary $\mathcal{A}$ corrupts servers before the start of the protocol². For an *adaptive corruption*, the adversary can corrupt servers at any time during the payment transaction.

2.3. Fault Flexibility

We allow clients to make two types of fault assumptions among the servers: *crash faults* and *Byzantine faults*. We make no assumptions on the number of faulty clients but provide security guarantees only for non-faulty clients. The clients assume that a PPT adversary $\mathcal{A}$ controls and coordinates all faulty nodes.

2.4. Byzantine Message Passing Model

Non-faulty nodes trigger *operations* in the system. To tolerate faulty nodes that may not respect the interfaces provided by the system, we assume but omit explicit references to the following in our system: (i) the existence of an authentication mechanism to ensure that faulty nodes cannot trigger operations on behalf of non-faulty nodes, and (ii) replay-protection mechanism using cryptographic hashes and sequence numbers.

Object Types. We define a sequential object type obj as a tuple $\text{obj} := (Q, q_0, \text{op}, \text{resp}, \mathcal{T})$, where Q is the set of all states of the object, q_0 is the initial state of the object, op is the set of all operations defined on the object, resp is the set of responses, and $\mathcal{T} : Q \times \mathcal{N} \times \text{op} \times \text{resp} \times Q \rightarrow \{0, 1\}$ is a validation relation that maps a prior state for a node and

1. Technically, partial synchrony consists of two types: Type-I where the network has a bounded delay Δ but the Δ bound is unknown to the nodes, and Type-II which we have defined. The two models are equivalent in the sense that we can convert protocols designed in one model into protocols in the other model, but we must note that in general, the same protocols do not work in both models. However, our protocols work in both the models.

2. A static adversary in the literature (e.g., [17]) corrupts all faulty nodes before the start of the protocol. For a long running system, this might appear impractical. For our application, we require a weaker assumption. Our static corruption assumption only requires that the adversary corrupts nodes before the receiver initiates the payment, i.e., sends the message containing the payment to the servers.

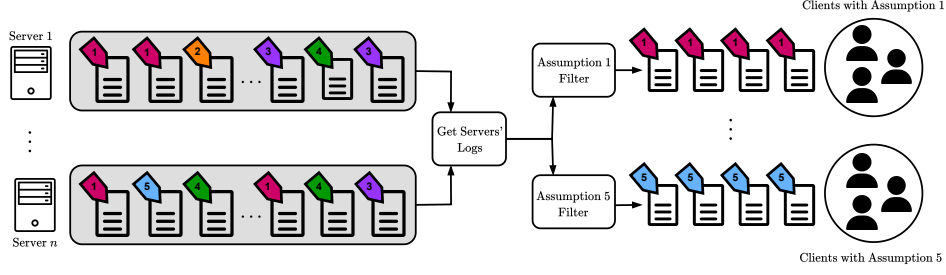


Figure 1. A logical overview of our system. The system consists of n servers and clients. The clients are logically grouped together based on their assumptions. We assign colors to these groups. The clients make payments based on their assumptions and the servers record them. Note that the servers maintain potentially consistent views; clients interact with the servers based on their assumptions to derive consistent views.

an operation to the resulting state and response as a valid transition.

Histories. A *history* is a sequence of invocations and responses. We associate each invocation or response with a node identifier. A *sequential history* is a history that has an invocation at the beginning, and every invocation is immediately followed by a response associated with the same node identifier. A sequential history is *legal* if its invocations and responses respect the validation relation $\mathcal{T}$ for some sequence of state assignments on the objects.

Implementations. We assume that the nodes (servers and clients) communicate by invoking atomic operations on shared objects. The abstract shared objects are *implemented* by the servers using an internal distributed protocol for every operation. We assume that non-faulty clients are sequential, i.e., they do not invoke a second operation before receiving a response for the first operation. We ensure security only for non-faulty clients.³ An *execution* of an implementation is a sequence of invocations and responses of operations of atomic accesses of the shared object instances.

A *projection* of a history H to node n_i , denoted by $H|n_i$ is the sub-sequence of H with label n_i . We say that an invocation in a history H by a node n_i is *incomplete* if it has no associated response in $H|n_i$. We say that a history is *complete* if it has no incomplete invocations for any node. A *completion* of a history H denoted by $\bar{H}$ consists of invocations and matching responses for all complete invocations in H . We can obtain a completion of a history by removing incomplete invocations or adding matching responses.

We denote the *ordering* between invocations and responses in a history H with $\prec_H$. An invocation op_i *precedes* an invocation op_{i+1} in H , denoted by $op_i \prec_H op_{i+1}$, if op_i is complete and the response precedes op_{i+1} in H .

For every possible access to the shared objects by the nodes, its execution results in an associated history which is a sequence of invocations and responses labelled with node identifiers, instance labels, and sequence numbers.

Sequential Consistency. Informally, an execution E with history H of an implementation of a shared object is *sequentially consistent* if there exists a completion $\bar{H}$ and

each node $n_i \in \mathcal{N}$ can explain its view $\bar{H}|n_i$ with some sequential history S_i .

Definition 2.1 (Sequential Consistency [20]). *Let $\mathcal{O}$ be a shared object of type obj . Let $ops(H) \in op^*$ denote the set of operations executed by non-faulty nodes in the history H . A sequentially consistent implementation of shared object $\mathcal{O}$ ensures that for any execution E with history H , there exists a completion $\bar{H}$, and for every non-faulty node n_i , a legal sequential history S_i exists such that:*

- $\bar{H}|n_i = S_i|n_i$, and
- $ops(\bar{H}) \subseteq ops(S_i)$

Sequential consistency allows nodes to locally reason about their history. However, it does not specify any constraints on the ordering of operations across nodes. For example, consider two non-faulty nodes n_i and n_j invoking operations one after the other. Definition 2.1 does not specify an ordering for the operations. The respective logs of n_i and n_j might be consistent according to sequential histories S_i and S_j respectively, but n_i 's log may not be consistent according to S_j , and vice versa.

Let H_1 and H_2 be two histories over an execution E . Let $\prec_H^1$ and $\prec_H^2$ be the partial orderings defined over H_1 and H_2 respectively. If operation op_1 precedes op_2 in H_1 according to the ordering $\prec_H^1$, and op_1 precedes op_2 in $\prec_H^2$ according to the ordering $\prec_H^2$, then we say that $\prec_H^1 \subseteq \prec_H^2$.

Linearizability. In order to specify a meaningful *correctness* in the ordering properties, we present the stronger, linearizability, specification from the literature which ensures that the operations are consistent for all non-faulty nodes.

Definition 2.2 (Linearizability [21], [7]). *A linearizable implementation of shared object $\mathcal{O}$ of type obj ensures that for any execution E with history H , there exists a completion $\bar{H}$ and a sequential history S such that:*

- 1) for all nodes $n_i \in \mathcal{N}$, $\bar{H}|n_i = S|n_i$;
- 2) $\prec_H \subseteq \prec_S$

In comparison to sequential consistency, linearizability ensures the existence of a single sequential history S that can explain the history for all the nodes in the system.

3. We assume sequential clients as this is a requirement for ensuring linearizability [18], [19].

3. Flexible Payment System Model

3.1. Account Objects

We use the abstraction of an *account object type* that tracks the account of a user in our payment system. We define an account object implementation A as follows:

- *State* (Q). The state consists of a set of accounts. We associate each account with an immutable value called the *owner*, and a natural number indicating the balance. We will use the struct notation $A.bal$ and $A.owner$ to denote these values for an account A .

- *Initial State* (q_0). The initial state consists of accounts specifying the owner and an initial balance.

- *Operations* (op). The account object implementation provides two operations: Balance and Transfer.

The balance operation Balance returns the current balance of the account. For accounts owned by faulty users, we allow the operation to output $\perp$. In this case, we say that the balance operation Balance has failed.

The transfer operation Transfer, takes as input the identifier of another account object implementation, and a transfer value. It returns 1 if the transfer is successful, and returns $\perp$ otherwise.

- *Transition Function* (T) and *Responses* ($resp$). For an account object implementation A , a transition from state Q_i , for node n_i , with operation op resulting in response $resp$ and state Q_{i+1} is valid if:

- *Correct Balance*. $op = \text{Balance}$ and $Q_i = Q_{i+1}$ and $resp = A.bal$. This transition ensures that if a balance operation is successful, then the states do not change, and the response is the balance as per the current state.

- *Failed Balance*. $op = \text{Balance}$ and $Q_i = Q_{i+1}$ and $resp = \perp$ and $A.owner$ belongs to a faulty user. This transition ensures that if a balance operation fails, then the states do not change and only accounts owned by faulty users can ever have a failed balance.

- *Correct Transfer*. $op = \text{Transfer}(A, A', v)$, $Q_{i+1}.A.bal = Q_i.A.bal - v$, $Q_{i+1}.A'.bal = Q_i.A'.bal + v$, and $resp = 1$. This transition ensures that if a transfer operation is successful ($resp = 1$), then the payer account A loses v value in the new state, the payee account gains v value in the new state.

- *Failed Transfer*. $op = \text{Transfer}(A, A', v)$ and $Q_i = Q_{i+1}$ and $resp = \perp$. This transition ensures that if a transfer operation fails ($resp = \perp$), then the states do not change.

3.2. Payment System

Given access to the account object types, we can define a payment system using the Byzantine message passing model.

On a high-level, linearizability is a strong property to ensure for all the operations of an account object [22]. For a generic state machine, it is well known that in order to ensure linearizability for operations that can be invoked by any node, no matter their result, we need to implement total order for the operations. Total ordering is expensive, compared to partial ordering or relaxed ordering implementations. To illustrate, gossiping a message to a majority of

the servers in Bitcoin is much more efficient than the Bitcoin consensus protocol.

For payment systems, Guerraoui et al. [7] noted that we can relax some of the components to avoid implementing a total ordering. We relax the following components:

- *We only need to ensure linearizability for successful transfers*. The correctness of a successful transfer is defined recursively over the correctness of incoming payments for the account initiating the transfer. Linearizability ensures that in an execution, we can find definite and permanent points in time called *linearization points* where the system executes the operations. Thus, requiring that all successful payments have such definite and permanent (non-revertible or reorder-able) points is necessary. Additionally, we want to ensure that non-faulty nodes can always pay others, and if they receive a payment, i.e., they consider a transfer as successful, then they can always spend it. As a result, when a non-faulty node i pays another node j , node i needs a guarantee that any node querying the balance of node i will see a balance at least the amount in the payment. This ensures that the new payment is always successful. Thus, we want to ensure linearizability for successful transfer operations.

- *We only need to ensure sequential consistency for the balance operations*. Sequential consistency [23] intuitively means that from the perspective of a node, the operations appear to be in some total order. For instance, let the account object be a sequentially consistent implementation. The object guarantees that a node i calling Balance on its own account without performing any Transfer will view the last known balance or a higher balance due to an ongoing incoming payment. At the same time, another node j may view older balances, even if they query the balance after node i , and this does not affect the correctness of the balance operation. Sequential consistency, as opposed to linearizability, relaxes the requirements of consistency to apply only locally, without guarantees for the states of other nodes. Thus, to ensure correctness of the Balance operation, we do not require linearizability.

- *Failed transfers only need sequential consistency*. There are two cases here: (i) failure of a payment from a faulty node, and (ii) failure of a payment from a non-faulty node. For the former case, when a transfer operation fails, every non-faulty node interacting with it can locally reason about the failure and thus discard the payment. For the latter case, since a non-faulty node initiated a payment, it must have ensured that it has sufficient balance to make payments. Now, there are two further cases: (a) no node observes that this payment was successful, or (b) at least one node observes that this payment was successful. Since the node that initiated the payment was non-faulty, it must have ensured that it has sufficient balance to make the payment. This balance is due to the incoming payments that it has observed. Since successful payments are linearizable, both of these cases are not possible.

Putting these ideas together, we formally define a payment system in Definition 3.1 as follows:

Definition 3.1 (Payment System). Let Acc be the set of accounts objects. Let E be any execution of a payment system implementing Acc account objects, and H be the corresponding history. Let $\text{ops}(H)$ denote the set of operations in history H executed by non-faulty nodes. A system implements the payment system abstraction for all accounts in Acc if there exists $\bar{H}$ a completion of H while ensuring the following:

- **Termination.** If a non-faulty client invokes the balance, Transfer or transfer operations, then the operations eventually terminate.

- **Correctness.** Let $\text{ops}^T(H)$ denote the set of successful transfer operations. Let $\bar{H}^T$ denote the sub-history of successful transfers of $\bar{H}$ performed by non-faulty nodes. Let $\prec_{\bar{H}}$ be an ordering over the history $\bar{H}$, and let $\prec_{\bar{H}}^T$ denote a subset of $\prec_{\bar{H}}$ restricted to operations in $\bar{H}^T$. There exists a legal sequential history S such that:

- for every non-faulty node n_i , $\bar{H}^T|n_i = S|n_i$, and
- $\prec_{\bar{H}}^T \subseteq \prec_S$

- **Soundness.** For every non-faulty node $n_i \in \mathcal{N}$ there exists a legal sequential history S_i such that:

- $\text{ops}(\bar{H}) \subseteq \text{ops}(S_i)$
- $S_i|n_i = \bar{H}|n_i$

Note that soundness requires sequential consistency (Definition 2.1) for all operations, and correctness requires linearizability (Definition 2.2) for successful transfers as output by non-faulty nodes.

We first show that this definition of a payment system ensures classical correctness properties for a payment system, i.e., the total amount of money in the system is conserved, and non-faulty users can spend their money.

Theorem 3.1. Assume a payment system satisfies Definition 3.1 and an authentication mechanism that authenticates the owner of an account for a transfer operation. Then, the payment system ensures the following:

- 1) non-faulty users can spend their money, and
- 2) total amount of money in the system is conserved.

Proof. Consider a payment system that satisfies Definition 3.1. Let E be any execution of an implementation and H be the corresponding history.

Non-faulty users can spend their money. The authentication mechanism ensures that only the owner of an account can spend the money in the account. Thus, only the owner can initiate a transfer operation. Let n be a non-faulty node initiating a transfer operation. From the termination property of the payment system, we know that the operation eventually terminates. Let n ensure that it has sufficient balance before initiating a transfer operation. In this case, n initiated a balance operation before the transfer operation. From the correctness property of the payment system, we know that there exists a legal sequential history S such that for every non-faulty node n_i , $\bar{H}^T|n_i = S|n_i$, where $\bar{H}^T$ is the sub-history of successful transfers of $\bar{H}$. In this view for n , the balance operation event precedes the transfer operation event. Thus, all non-faulty nodes observe that the

balance operation before the transfer operation. Since n is non-faulty, it does not initiate any other concurrent transfer operations (from the assumption that all non-faulty nodes are sequential). Thus, all non-faulty nodes observe that n has sufficient balance before initiating a transfer operation. As a result, all non-faulty nodes result in a successful transfer operation. The soundness property ensures that if a non-faulty node observes a failed incoming transfer operation, then it can reason about it using a sequential history S_i . Thus, non-faulty users can always spend their money.

Conservation of Money. The total money in the system can increase only if there is partial execution of the transfer operation. From the correctness property of the payment system, we know that all successful transfers are linearizable and thus all transfer operations are executed atomically. $\square$

3.3. Hybrid Consistent Broadcast

A (Byzantine) Consistent Broadcast (BCB) primitive [10] is a broadcast primitive that ensures that it only outputs one consistent message to all the nodes participating in the protocol when the broadcaster is non-faulty. Since our system consists of two classes of nodes: *servers* and *clients*. The clients play a passive role and only initiate the broadcast of the transfer operations. Our payment system requires that a non-faulty client processes a payment only once. This demands that the primitive ensures that all clients output at most one payment message. The servers implement the consistent broadcast allowing all clients to output a linearizable view of the system state.

We start with the definition of Hybrid Consistent Broadcast [13] (HBCB) that captures our two classes of nodes. This primitive consists of two operations: $\text{broadcast}_s(m)$ and $\text{deliver}_i(m, s)$ where s and i are labels for nodes, and m is an arbitrary message. A hybrid BCB primitive consists of two sets of nodes: a sender set $\mathcal{S}$ that call the broadcast operation, and a receiver set $\mathcal{R}$ that call the deliver operation. Along with other properties, the primitive ensures that the deliver operation outputs the same (consistent) value to all nodes in the receiver set.

Extending the original definition, we require the use of a hybrid BCB primitive that ensures that all non-faulty clients output the same order of messages from the same sender. To achieve this, we augment the hybrid BCB primitive with an additional property called the *Source Order (SO)* property. This property ensures that non-faulty clients output the messages in the same order. Looking ahead, we can realize this by augmenting an implementation of HBCB with counters.

We present our definition of hybrid BCB with Source Ordering (SO) as follows:

Definition 3.2 (Hybrid Consistent Broadcast with Source Order (HBCB-SO)). Define $\mathcal{S}$ as the set of sender nodes and $\mathcal{R}$ as the set of delivery nodes. A Hybrid-Consistent Broadcast instance hbcbs-so with senders $s \in \mathcal{S}$ and delivery nodes $\mathcal{R}$ consists of two events: (i) $\text{hbcbs-so.broadcast}_s(m)$: executed by a sender node $s \in \mathcal{S}$, and (ii) $\text{hbcbs-so.deliver}_i(m, s)$ executed by a delivery node

$r_i \in \mathcal{R}$ for sender $s \in \mathcal{S}$. We say that a sender broadcasts message m if it calls $hcbcb\text{-}so.\text{broadcast}_s(m)$, and similarly with $hcbcb\text{-}so.\text{deliver}_i(m, s)$ we say that a non-faulty delivery node $r_i \in \mathcal{R}$ delivers message m from the sender $s \in \mathcal{S}$. The instance ensures the following properties:

- **Validity.** If a non-faulty sender node $s \in \mathcal{S}$ broadcasts a message m , then every non-faulty delivery node $r_i \in \mathcal{R}$ eventually delivers m .
- **No Duplication.** Every non-faulty delivery node $r_i \in \mathcal{R}$ delivers at most one message.⁴
- **Integrity.** If some non-faulty delivery node delivers a message m for sender node $s \in \mathcal{S}$ and sender node s is non-faulty, then m was previously broadcast by s .
- **Consistency.** If some non-faulty delivery node $r_i \in \mathcal{R}$ delivers m and another non-faulty delivery node $r_j \in \mathcal{R}$ delivers m' , then $m = m'$.
- **Source Order (SO).** If two non-faulty nodes $r_i, r_j \in \mathcal{R}$ both deliver messages m and m' from a sender $s_i \in \mathcal{S}$, then r_i delivers m before m' if s_i broadcasts m before m' .

Looking ahead, our motivation for Definition 3.2 is that the clients form the external set of senders $\mathcal{S} \leftarrow \mathcal{C}$ and the delivery also occurs at the clients, i.e., $\mathcal{R} \leftarrow \mathcal{C}$. The implementation uses the servers to achieve the properties of $hcbcb\text{-}so$. The source order property ensures that if a non-faulty client performs a payment based on some dependent payments, all other non-faulty clients observe the same sequence of dependent payments thus allowing all payments to be successful.

3.4. Payment System using HBCB-SO

We present an implementation of a payment system (Definition 3.1) in Fig. 2 using a black box implementation of an account object and HBCB-SO.

Clients maintain local logs of transactions in *hist* (Line 2). This allows them to query balances of other clients without interacting with the servers. Initially, we assume that all accounts have some non-negative integer balance.

Let $hcbcb\text{-}so$ be an implementation of hybrid consistent broadcast with SO, with the sender and delivery nodes as the set of clients, i.e., $\mathcal{S} = \mathcal{R} = \mathcal{C}$. Consider a scenario where Alice owning account A wants to pay Bob owning account A' a value of v amount. Alice generates a payment instruction consisting of: (i) Bob's account identifier, (ii) the value v , (iii) a tag Pay (Line 11), and (iv) a set consisting of incoming payments that were processed before the last successful outgoing payment was broadcast. Alice creates a digital signature on the previous four values that authenticates the payment which we define as an *authorization*.

At this point, technically, both Alice or Bob can initiate the broadcast of the payment instruction along with the authorization. WLOG, in this section we assume that the payer, i.e., Alice initiates the broadcast.

The $hcbcb\text{-}so$ instance using the servers ensures that all clients output the same order of transactions from a client.

4. The original version is incorrect in defining the *No duplication* property by stating the delivery requirement for $\mathcal{S}$. We present the corrected version by specifying it for the nodes in the delivery set $\mathcal{R}$.

When $hcbcb\text{-}so$ delivers a payment instruction to the clients, they add it to a local queue. When a payment satisfies the following conditions, it is processed by the clients: (i) the authorization (Line 17), and (ii) the dependent transactions are already processed by the client. After ensuring that a payment satisfies the above conditions, the clients check whether Alice's account A has sufficient balance (Line 25).

All clients maintain the set of incoming payments using the data structure *lastIn*. Thus, when Bob processes the payment transaction, he adds the payment transaction to *lastIn*. Finally, all clients add the payment transaction to the sender and receiver's history.

Security Analysis. Due to space constraints, we defer the full proofs to Section D. The intuition for the security is as follows. When Alice creates a payment, she references all the previous incoming payments since the last successful payment in the message component *lastIn*. This creates a partially ordered graph of transfer operations. Due to the consistency properties of $hcbcb\text{-}so$, all non-faulty nodes view the same partially ordered sequence of transfers. Since Alice is non-faulty, the references in *lastIn* were successful transfer operations for Alice.

4. FPS – Flexible Payment System

Overview. We showed how to *implement* the payment system abstraction (from Definition 3.1) using an HBCB-SO instance. In this section, we show how to implement an instance of the HBCB-SO using servers so that clients with different assumptions can interpret messages to implement a hybrid BCB with SO among all clients with the same assumptions.

4.1. Modeling Flexible Clients

A *flexible* client is a client within the system that submits a payment *transaction* to the servers with flexible assumptions. Each flexible client defines its own assumptions regarding the server network, the client-server network, the types and number of faults among servers, and the fault corruption mechanism. We use a three-letter code to represent the assumptions of a flexible client, which we summarize in Table 1. An *NCF*-flexible client is one that can choose among the following:

- **Network (N).** The client selects between two network assumptions: (i) the network between servers is Δ -bounded synchronous, and the client-server network is Δ_C -bounded synchronous (N_S), or (ii) both the inter-server network and the client-server network are Δ -partial synchronous (N_P).
- **Corruption (C).** The clients assume that an adversary $\mathcal{A}$, modeled as a probabilistic polynomial-time (PPT) entity, can corrupt a server, obtaining its internal state and full control. Server corruption may be either (i) static (C_S) or (ii) adaptive (C_A).
- **Fault (F).** The clients assume that the system can experience one of the two fault types: (i) t_c crash faults (F_C), or (ii) t_b Byzantine faults (F_B).

Local Variables <pre> 1: queue $\leftarrow \emptyset$ 2: hist[A] $\leftarrow \emptyset$: $\forall A \in \text{Acc}$ 3: lastIn $\leftarrow \emptyset$ 4: hbcB-so: $S \leftarrow C, \mathcal{R} \leftarrow C$ 5: operation Balance(A, txs) 6: return sum of incoming and outgoing payments in txs 7: operation Transfer(A, A', v) 8: txs $\leftarrow \text{hist}[A] \cup \text{lastIn}$ 9: if Balance(A, txs) < v then return $\perp$ 10: msg.data $\leftarrow (A, A', v, \text{lastIn})$ // c_i is the owner of A 11: msg.tag $\leftarrow \text{Pay}$ 12: msg.auth $\leftarrow \text{Sign}(\text{msg.data})$ 13: hbcB-so.broadcast(msg) 14: lastIn $\leftarrow \emptyset$ 15: function VALIDATE(msg) </pre>	<pre> 16: Parse msg.data := (A, A', ·, lastInMsg) 17: if msg.auth is invalid then return $\perp$ 18: if lastInMsg $\not\subseteq \text{hist}[A]$ then return $\perp$ 19: return 1 20: on hbcB-so.deliver_i(msg, c_i) 21: queue $\leftarrow \text{queue} \cup \{\text{msg}\}$ 22: on msg $\in \text{queue} \wedge \text{VALIDATE}(\text{msg}) = 1$ 23: queue $\leftarrow \text{queue} \setminus \{\text{msg}\}$ 24: Parse msg.data := (A, A', v, ·) 25: if Balance(A, hist[A]) < v then 26: return $\perp$ 27: if c_i owns A' then 28: lastIn $\leftarrow \text{lastIn} \cup \{\text{msg}\}$ 29: hist[A] $\leftarrow \text{hist}[A] \cup \{\text{msg}\}$ 30: hist[A'] $\leftarrow \text{hist}[A'] \cup \{\text{msg}\}$ 31: return 1 </pre>
--	---

Figure 2. Algorithms implementing a payment system using a hybrid-BCB instance.

For instance, an $N_P C_A F_B$ -client assumes that the network is partially synchronous, the corruption is adaptive, and the system contains Byzantine faults.

When describing algorithms, we use the notation N_* to denote any network assumption, C_* to denote any corruption assumption and F_* to denote any fault assumption. We say that a non-faulty client makes a *correct* assumption if the assumption in reality matches the assumption made by the client.

TABLE 1. CODIFYING THE FLEXIBLE ASSUMPTIONS OF AN NCF -FLEXIBLE CLIENT. N_* , C_* , F_* ARE RESPECTIVELY THE NETWORK, CORRUPTION, AND FAULT ASSUMPTIONS.

Label	Parameter	Assumption
N_S	Δ_S, Δ_C	Bounded Synchronous
N_P	—	Partially Synchronous
C_S	κ	Static Corruption
C_A	—	Adaptive Corruption
F_C	t	Crash Faults
F_B	t	Byzantine Faults

4.2. FPS System details

In this section, we will present the full FPS system that implements the payment system (Definition 3.1) simultaneously for all assumptions.

Receiver-centric payments. In the previous section, we mentioned that both Alice and Bob can drive the transfer operation to completion. For our system, we assume that Alice signs the payment instruction, which we refer to as a *payment transaction* and provides the signed payment transaction to the payee, Bob. Here, Bob broadcasts the payment transaction to the system based on his assumptions, determining the success of the payment.

Assumption Registration. The system operates in epochs, numbered sequentially as $1, 2, \dots, e$. At the beginning of each epoch, clients have the option to *register* their assumptions; if a client does not register any assumptions, the system assigns a default assumption. Although we do not cover implementation specifics here, the system triggers epoch changes periodically or in response to specific

conditions. During an epoch change, all servers engage in a protocol to reach consistency in assumptions across all clients. A system can achieve this agreement through a consensus protocol or by an external authority overseeing the system (see Section B).

```

1: function MSG(sk, tag, data)
2:  msg.from  $\leftarrow n_i$ 
3:  msg.tag  $\leftarrow \text{tag}$ 
4:  msg.data  $\leftarrow \text{data}$ 
5:  msg.auth  $\leftarrow \text{Sign}(\text{sk}, (\text{tag}, \text{data}))$ 
6:  return msg
7: function QC(tag, data, msgs, size)
8:  if |msgs| < size then return  $\perp$ 
9:  for m  $\in \text{msgs}$  do
10:   if m.tag  $\neq \text{tag}$  then return  $\perp$ 
11:   if m.data  $\neq \text{data}$  then return  $\perp$ 
12:   if m.auth is not valid then return  $\perp$ 
13:  return 1
14: function CREATEPAYMENT( $A_s, A_r, v, N_* C_* F_*, \text{sk}, \text{sn}$ )
15:  payment.from  $\leftarrow A_s$ 
16:  payment.to  $\leftarrow A_r$ 
17:  payment.value  $\leftarrow v$ 
18:  payment.assumption  $\leftarrow N_* C_* F_*$ 
19:  payment.auth  $\leftarrow \text{Sign}(\text{sk}, (A_s, A_r, v, \text{sn}))$ 
20:  payment.sn  $\leftarrow \text{sn}$ 
21:  return payment
22: function RANDQ( $\mathcal{N}, r_t, \kappa$ )
23:  return a random subset of size  $\ell$  from  $\mathcal{N}$  such that the
24:  probability of  $\lceil r_t \cdot \ell \rceil$  servers being faulty is  $\text{negl}(\kappa)$ 

```

Figure 3. FPS utility functions

Life-cycle of a flexible transfer. Consider a payment between Alice and Bob for value v . Figure 4 provides a technical overview of the life cycle of a payment within the FPS protocol.

For any two non-faulty Alice paying Bob, with both parties having the same assumptions, Alice having a correct assumption and sufficient balance, our protocol ensures that Alice will always be able to pay Bob successfully. Therefore, initially, Alice and Bob need to agree on a set of assumptions. If their assumptions differ but are compatible (Section B), Alice can safely proceed. Compatibility means payment flows from stronger to weaker assumptions (e.g., Alice tolerates more faults than Bob). However, it's not safe for Alice to receive a payment from Bob since the reverse

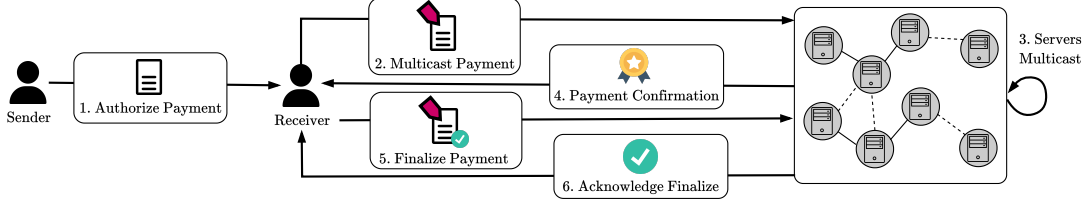


Figure 4. A technical overview of FPS.

is not true.

All clients maintain a sequence number initialized to 0. When Alice wants to pay Bob, she creates a payment transaction using the `CREATEPAYMENT` function (Fig. 3) and provides the payment transaction tx to Bob. This function takes as input sk the secret key associated with the sending account A_s , the sending and receiver accounts (A_s , A_r), the latest sequence number for the sender, and the amount (v) that the sender wants to transfer to the recipient. Alice then sends the authorized payment obtained from the `PAY` (Fig. 6) to Bob.

On receiving an authorized payment transaction $paymsg$, Bob checks if it is in the correct format, and that Alice has used its account A_r as the recipient of v amount. It then checks that the signature on it is valid.

Since Bob does not know all of Alice’s transactions, he cannot check the validity of the sequence number sn . This is where Bob uses assistance from servers. Bob wants to ensure that the state as per transactions made by Alice as per the transactions with sequence numbers up to sn , Alice has sufficient balance and this is the only payment transaction that the system accepts as a valid transaction.

If the payment transaction is locally valid at the servers, (i.e., Alice’s account has sufficient funds and the sequence number is correct), the servers respond with signed acknowledgments for the transaction. Based on the assumptions, Bob collects sufficient acknowledgments and/or waits for any necessary timeouts before *confirming* the payment as final.

If the payment transaction is locally inconsistent, the servers return an error message to Bob who can then interpret and respond to the message based on their assumptions.

FPS Protocol. Our protocol is organized into atomic **On** conditions, executed by the servers when these conditions are met. We use utility functions described in Fig. 3 to detail our algorithms. The function `MSG` encodes messages with metadata, including the source (*from*), message type (*tag*), payload (*data*), and sender authorization using a digital signature (*auth*). Additionally, the `QC` helper function checks if a set of message $msgs$ constitutes a quorum certificate of type *tag* and quorum size *size*. Alice (the payer) creates the payment transaction using the `CREATEPAYMENT` utility function. To support flexible clients with static assumptions, we use `RANDQ` to select a random set of nodes from $\mathcal{N}$, with an allowable fault ratio r_t . This ensures that the probability of having more than $r_t \cdot |\mathcal{N}|$ faulty nodes remains negligible, given the security parameter κ .

4.3. Server-side protocol

To implement a flexible payment system, it is essential that the server-side protocol avoids embedding any assumptions regarding system parameters like the number of faults, type of corruption, or network behavior. This separation prevents the servers from relying on quorum certificates, enabling flexible clients to interpret the protocol according to their own assumptions.

Our design’s central concept is the use of consistent broadcast for payment transactions while keeping quorum certificates separate from the payment protocol.

In Fig. 5, we outline our server-side protocol aligned with this approach. Servers receive payment transactions agnostic to their origin, which could be from the sender, the recipient, or relayed through multicast by other servers. Each server independently validates transactions using its local state, checking: (i) digital signature authenticity, (ii) sufficient sender balance, (iii) correct sequence number ($sn = sn[A_s] + 1$), and (iv) the transaction has not been previously processed ($status[A_s, sn] = \perp$). During validation, if a server has already processed a different payment with the same sequence number (equivocation), it sends an error message to the recipient, enabling synchronous clients to detect double-spending attempts during their timeout period.

Servers handle payments differently based on the recipient’s assumption:

- **Bounded synchronous recipients:** The server immediately applies the transaction (debiting sender, crediting recipient) and marks it as *Confirmed*. This enables single-round finalization under synchrony assumptions.
- **Partially synchronous recipients:** The server *locks* the transaction without applying it, preventing conflicting payments with the same sequence number. The transaction is applied only upon finalization.

All servers multicast validated payments to ensure consistency. During finalization, servers verify the quorum certificate matches the locked or confirmed payment. If equivocation is detected during finalization, servers send error messages to the recipient while preserving the first observed payment to prevent Byzantine senders from reverting valid transactions. Thus, equivocation detection occurs in both the payment validation phase (to enable synchronous timeout-based detection) and the finalization phase (to catch conflicts that arise from network delays).

Servers also respond to balance queries, returning the current balance which reflects only finalized transactions for partially synchronous payments.

<pre> 1: State 2: $balance : A \rightarrow \mathbb{N}$ // Map accounts to balances 3: $sn : A \rightarrow \mathbb{N}$ // Map accounts to sequence numbers 4: $status : (A, \mathbb{N}) \rightarrow \{\perp, Locked(tx), Confirmed(tx), Finalized(tx)\}$ // Assumption registration ensures sender uses consistent recipient assumptions // Retry mechanisms handle out-of-order finalize messages 5: on $msg \wedge VALIDPAY(msg) = 1$ 6: Parse $msg := MSG(\cdot, pay)$ 7: Parse $pay := (A_s, A_r, v, \sigma, sn')$ 8: if A_r has assumption N_S then 9: APPLYTX(pay) 10: $status[A_s, sn'] \leftarrow Confirmed(pay)$ 11: else if A_r has assumption N_P then 12: $status[A_s, sn'] \leftarrow Locked(pay)$ 13: $msgecho \leftarrow MSG(\cdot, Echo, pay)$ 14: multicast $msgecho$ 15: $msgconfirm \leftarrow MSG(\cdot, Confirm, pay)$ 16: Send $msgconfirm$ to A_s and A_r 17: on $msg \wedge VALIDFIN(msg) = 1$ 18: Parse $msg := MSG(Fin, (pay, qc))$ 19: Parse $pay := (A_s, A_r, v, \sigma, sn')$ 20: $s \leftarrow status[A_s, sn']$ 21: if A_r has assumption N_S then 22: if $s = Confirmed(pay')$ then 23: if $pay = pay'$ then 24: $status[A_s, sn'] \leftarrow Finalized(pay)$ 25: Send $MSG(\cdot, Fin, pay)$ to A_r 26: else // Equivocation detected 27: Send $MSG(\cdot, Error, (pay, pay'))$ to A_r 28: else if $s = Finalized(pay')$ then 29: if $pay \neq pay'$ then 30: Send $MSG(\cdot, Error, (pay, pay'))$ to A_r 31: else 32: Send $MSG(\cdot, Fin, pay)$ to A_r 33: return 34: if A_r has assumption N_P then 35: if $s = Locked(pay')$ then 36: if $pay = pay'$ then 37: APPLYTX(pay) 38: $status[A_s, sn'] \leftarrow Finalized(pay)$ 39: Send $MSG(\cdot, Fin, pay)$ to A_r </pre>	<pre> 40: else // Equivocation - warn but don't apply 41: Send $MSG(\cdot, Error, (pay, pay'))$ to A_r 42: else if $s = Finalized(pay')$ then 43: if $pay \neq pay'$ then 44: Send $MSG(\cdot, Error, (pay, pay'))$ to A_r 45: else 46: Send $MSG(\cdot, Fin, pay)$ to A_r 47: return 48: on $msg.tag = BalQuery$ 49: Parse $msg.data := A$ 50: $bal \leftarrow balance[A]$ 51: Send $MSG(\cdot, BalResp, (A, bal))$ to sender of msg 52: function APPLYTX(pay) 53: Parse $pay := (A_s, A_r, v, \cdot, \cdot)$ 54: $balance[A_s] \leftarrow balance[A_s] - v$ 55: $balance[A_r] \leftarrow balance[A_r] + v$ 56: $sn[A_s] \leftarrow sn[A_s] + 1$ 57: function VALIDPAY(msg) 58: Parse $msg := MSG(tag, pay)$ 59: Parse $pay := (A_s, A_r, v, \sigma, sn')$ 60: $s \leftarrow status[A_s, sn']$ 61: // Check for equivocation and warn recipient 62: if $s = Locked(pay') \vee s = Confirmed(pay') \vee s = Finalized(pay')$ then 63: if $pay \neq pay'$ then 64: Send $MSG(\cdot, Error, (pay, pay'))$ to A_r 65: return $tag \in \{Pay, Echo\} \wedge$ 66: σ is valid w.r.t. $A_s \wedge$ 67: $sn' = sn[A_s] + 1 \wedge$ 68: $balance[A_s] \geq v \wedge$ 69: $status[A_s, sn'] = \perp$ 70: function VALIDFIN(msg) 71: Parse $msg := MSG(Fin, (pay, qc))$ 72: Parse $pay := (A_s, A_r, v, \sigma, sn')$ 73: $s \leftarrow status[A_s, sn']$ 74: $valid \leftarrow QC(Confirm, pay, qc, \cdot)$ 75: if A_r has assumption N_S then 76: $valid \leftarrow valid \wedge (s = Confirmed(\cdot) \vee s = Finalized(\cdot))$ 77: else if A_r has assumption N_P then 78: $valid \leftarrow valid \wedge (s = Locked(\cdot) \vee s = Finalized(\cdot))$ 79: return $valid$ </pre>
--	---

Figure 5. FPS payment protocol for servers

4.4. NCF-Flexible Client Protocol(s)

Using the server protocol described in Fig. 5, an NCF-flexible client makes the payment creating a payment (see Fig. 3). The signed transaction is given to the payment receiver who uses the algorithms in Figs. 6 and 7 and applies its assumptions and determines the finality of the payment.

We present intuitive description of the protocol and assumption algorithm specified in Figs. 6 and 7 now. Let us start by assuming that the corruption is adaptive.

Sequence numbers. We introduce sequence numbers in the system (but do not pollute our description with it) to ensure that the servers process the transactions from a sender in order. The sequence number results in the *source order* property and we only need to ensure consistency in the flexible hybrid BCB implementation.

Summary of flow. All clients send their payment transaction to a quorum of servers determined by their assumptions. Since all servers multicast the payment to other servers, this ensures all non-faulty servers eventually receive the transaction. Based on their network assumptions, clients follow either a single-round (bounded synchronous) or two-round (partially synchronous) protocol. Both protocols con-

sist of a confirmation phase followed by a finalization phase, differing in their timeout and quorum requirements.

Bounded Synchronous Assumptions. A bounded synchronous client (N_S, C_*, F_*) multicasts its payment to a quorum of $t + 1$ servers. The client waits for $t + 1$ confirmation acknowledgments, then waits for $2\Delta_C + \Delta_S$ to detect any equivocation errors. If no errors are observed, the client multicasts a finalization message with the quorum certificate to $t + 1$ servers (crash faults) or $2t + 1$ servers (Byzantine faults). After waiting another $2\Delta_C + \Delta_S$, the client collects $t + 1$ finalization acknowledgments to complete the payment. The fault-tolerance constraints are $t < n$ for crash faults and $t < n/2$ for Byzantine faults.

Partially Synchronous Assumptions. A partially synchronous client (N_P, C_*, F_*) multicasts its payment to $t + 1$ servers. The protocol consists of two phases: confirmation and finalization. In the confirmation phase, the client waits for acknowledgments from $\lceil n/2 \rceil + 1$ servers (crash faults) or $\lceil (n+t)/2 \rceil + 1$ servers (Byzantine faults). These quorum sizes ensure that any two quorums overlap in at least one non-faulty server, preventing double-spending.

Once the client obtains a quorum certificate, it multicasts

```

1: State
2:  $sn \leftarrow 0$ 
3: Assumption  $N_*, C_*, F_*$ 
4: function PAY( $A_s, A_r, value$ )
5:    $sk \leftarrow$  secret key associated with  $A_s$ 
6:    $paytx \leftarrow \text{CREATEPAYMENT}(A_s, A_r, value, N_*C_*F_*, sk, sn)$ 
7:    $sn \leftarrow sn + 1$ 
8:    $paymsg \leftarrow \text{MSG}(sk, \text{Pay}, paytx)$ 
9:   return  $paymsg$ 
10: function RECEIVE( $paymsg$ )
11:   Parse  $paymsg := \text{MSG}(\text{Pay}, pay)$ 
12:   Parse  $pay := (A_s, A_r, v, \sigma, txsn)$ 
13:   if  $\sigma$  for  $A_s$  is invalid then return  $\perp$ 
14:   if  $A_r$  and  $v$  are incorrect then return  $\perp$ 
15:    $Q \leftarrow \text{CONFIRMQUORUM}(C_*, F_*)$ 
16:   Multicast  $paymsg$  to servers in  $Q$ 
17:   ASSUMPTION.DECIDE( $pay$ )
18: function CONFIRMQUORUM( $C_*, F_*$ )
19:    $t \leftarrow$  fault threshold from assumption  $F_*$ 
20:   if  $C_* = C_A$  then
21:     return set of any  $t + 1$  servers
22:   else
23:     return  $\text{RANDQ}(\mathcal{N}, r_t, \kappa)$ 
24: function FINQUORUM( $N_*, C_*, F_*$ )
25:    $t \leftarrow$  fault threshold from assumption  $F_*$ 
26:   if  $C_* = C_A$  then
27:     if  $N_* = N_S$  then
28:       if  $F_* = F_C$  then return set of any  $t + 1$  servers
29:       if  $F_* = F_B$  then return set of any  $2t + 1$  servers
30:     else if  $N_* = N_P$  then
31:       return set of any  $t + 1$  servers
32:   else
33:     if  $N_* = N_S$  then
34:       return  $\text{RANDQ}(\mathcal{N}, r_t, \kappa)$ 
35:     else if  $N_* = N_P$  then
36:       return  $\text{RANDQ}(\mathcal{N}, r_t, \kappa)$ 
37: function BALANCEQUORUM( $N_*, C_*, F_*$ )
38:    $t \leftarrow$  fault threshold from assumption  $F_*$ 
39:   if  $C_* = C_A$  then
40:     if  $N_* = N_S$  then
41:       if  $F_* = F_C$  then return set of any  $t + 1$  servers
42:       if  $F_* = F_B$  then return set of all  $n$  servers
43:     else if  $N_* = N_P$  then
44:       return set of all  $n$  servers
45:   else
46:     return  $\text{RANDQ}(\mathcal{N}, r_t, \kappa)$ 

```

Figure 6. Client utilities: Payment creation, validation, and quorum selection

a finalization message with the certificate to $t + 1$ servers. This “uploads” the certificate to convince servers that may not have seen the original payment to lock or apply the transaction. The client then waits for $t + 1$ finalization acknowledgments to complete the payment. At this point, the payment is guaranteed to eventually finalize across all non-faulty servers.

The quorum sizes are chosen to prevent conflicting payments from being confirmed. For crash faults, a majority quorum ($\lceil n/2 \rceil + 1$) ensures that two conflicting payments cannot both obtain certificates, requiring $n > 2t$ for liveness. For Byzantine faults, the quorum size $\lceil (n + t)/2 \rceil + 1$ accounts for potential equivocation, ensuring that two quorums overlap in more than t servers, requiring $n > 3t$ for safety.

Static Corruptions. We can modify our protocol under the static corruption model, to use randomized quorums. Let κ be a statistical security parameter. We can find random quorums that ensure that at least one non-faulty server exists with non-negligible probability [25], at least half the

```

1: function ASSUMPTION.DECIDE( $pay$ )
2:    $t \leftarrow$  fault threshold from assumption  $F_*$ 
   // Client retries finalize if insufficient responses received
   // Bounded Synchrony
3:   if  $N_* = N_S$  then
4:     Wait for  $qc$  s.t.  $\text{QC}(\text{Confirm}, pay, qc, t + 1)$ 
5:     Wait  $2\Delta_c + \Delta_s$  for no equivocation errors
6:      $Q \leftarrow \text{FINQUORUM}(N_*, C_*, F_*)$ 
7:      $finmsg \leftarrow \text{MSG}(sk, \text{Fin}, (pay, qc))$ 
8:     Multicast  $finmsg$  to servers in  $Q$ 
9:     Wait  $2\Delta_c + \Delta_s$ 
10:    Wait for  $qc_{fin}$  s.t.  $\text{QC}(\text{Fin}, pay, qc_{fin}, t + 1)$ 
11:    return  $\top$ 
   // Partial Synchrony
12:   if  $N_* = N_P$  then
13:     if  $F_* = F_C$  then
14:       Wait for  $qc$  s.t.  $\text{QC}(\text{Confirm}, pay, qc, \lceil n/2 \rceil + 1)$ 
15:     else if  $F_* = F_B$  then
16:       Wait for  $qc$  s.t.  $\text{QC}(\text{Confirm}, pay, qc, \lceil (n + t)/2 \rceil + 1)$ 
17:      $Q \leftarrow \text{FINQUORUM}(N_*, C_*, F_*)$ 
18:      $finmsg \leftarrow \text{MSG}(sk, \text{Fin}, (pay, qc))$ 
19:     Multicast  $finmsg$  to servers in  $Q$ 
20:     Wait for  $qc_{fin}$  s.t.  $\text{QC}(\text{Fin}, pay, qc_{fin}, t + 1)$ 
21:     return  $\top$ 
22: function CHECKBALANCE( $A$ )
23:    $Q \leftarrow \text{BALANCEQUORUM}(N_*, C_*, F_*)$ 
24:    $balReq \leftarrow \text{MSG}(sk, \text{BalQuery}, A)$ 
25:   Multicast  $balReq$  to servers in  $Q$ 
26:   if  $N_* = N_S$  then
27:     Wait  $2\Delta_c + \Delta_s$ 
28:     Collect responses  $R$ 
29:      $t \leftarrow$  fault threshold from assumption  $F_*$ 
30:     if  $|R| \geq t + 1$  then
31:       return  $\min\{bal : (A, bal) \in R\}$ 
32:   else
33:     return  $\perp$  // Insufficient responses
34:   if  $N_* = N_P$  then
35:     Collect responses  $R$ 
36:     if  $F_* = F_C$  then
37:       Wait for  $\lceil n/2 \rceil + 1$  responses
38:     else if  $F_* = F_B$  then
39:        $t \leftarrow$  fault threshold from assumption
40:       Wait for  $\lceil (n + t)/2 \rceil + 1$  responses
41:     return  $\min\{bal : (A, bal) \in R\}$ 

```

Figure 7. Client protocol execution: Payment finalization and balance queries

quorum is non-faulty with non-negligible probability [26], [17], at least one-third of the quorum is non-faulty with non-negligible probability [26].

Summary of results. We show that Fig. 2 implements a payment system (Definition 3.1) when given access to a primitive *hbcbs* that satisfies the properties of hybrid Consistent Broadcast Definition 3.2. We present a summary of our results in Table 2.

TABLE 2. SUMMARY OF ADAPTIVE FAULT-TOLERANCE BOUNDS AND QUORUM SIZES.

Assumption	Bound	Quorum
N_SF_C	$n > t$	—
N_SF_B	$n > 2t$	—
N_PF_C	$n > 2t$	$n/2 + 1$
N_PF_B	$n > 3t$	$(n + t)/2 + 1$

Due to space constraints, we defer the full proofs to the appendix.

5. Evaluation

In this section, we evaluate the performance of our payment system.

Implementation details. We implemented an academic prototype of our proposed system with 33,793 LOC in Rust. We used EdDSA [27] as our signature scheme and SHA256 as the hash function. We added a counter to our transactions for replay protection. We will make our code and experiment logs available upon request (or public upon acceptance). We use RocksDB with default settings as our store and do not have in-memory data structures to save checkpoints on the disk and ensure recovery in the event of a crash. Our experiments do not focus on scalability of each assumption since prior works have already established that consensus-free payment systems are scalable [9].

Optimizations. We use hashes to refer to the transaction during finalization. We use the standard batching techniques to reduce cryptographic overheads. We use a batch size of 100 payments per payment request.

Setup. We use up to eight m5.large AWS instances with 8 RAM and 2 vCPUs. We use separate machines for every server and clients.

5.1. Micro Benchmarks

TABLE 3. MICRO BENCHMARK OF OUR SYSTEM. (S) REPRESENTS SERVERS, AND (C) REPRESENTS CLIENTS.

Name	Node	Size	Time
Tx Generation	(C)	88 B	0.273 ms
Tx Verif.	(S)	—	0.429 ms
Fin. Generation	(C)	257 + 128 <i>t</i> B	0.209 ms
Fin. Verif. (<i>t</i> = 1)	(S)	—	1.504 ms

We present some representative operations, their sizes, and times to compute them sequentially in Table 3. Our timing costs involve the time to write the intermediate states to the database. We observe that the transaction is an assumption-independent data structure, and thus, its generation and verification times are independent of the assumption. The finalization is assumption-specific as the number of signatures contained is a function of *t* and is only done when assuming partial synchrony. We observe that generating a finalization message costs a constant amount of time since the signatures are already verified when confirming the transaction, but their size is a function of *t*. On the other hand, the time to verify finalization is a function of *t* in two aspects: verifying the signature and the size of the data structures written to the disk.

The servers, despite being agnostic to the assumption, verify all the signatures in a certificate and thus perform asymmetric work despite being assumption-agnostic.

5.2. Assumption evaluation

General Methodology. We measure the number of payments per second of our payment system as throughput. We use $n = 4$ servers and clients. All clients use the same

assumptions. Since our n is small in our experiments, we use adaptive assumptions for the clients. When choosing a bounded synchronous network assumption, a key challenge for the client is to determine the values for Δ_S and Δ_C . For our experiments, we first measured the latency of partially synchronous payments (Byzantine and maximum faults), and used the 99% latency as the latency values for Δ_S and Δ_C . This value is $48.11 \approx 49$ ms.

Mixing Assumptions. In this section, we aim to measure the impact of assumptions on other clients with other assumptions.

Methodology. We measure the throughput and latency when the system is split between three configurations: (75%, 25%), (50%, 50%), and (25%, 75%) respectively of bounded sync. vs. partially synchronous Byzantine adaptive assumptions. Fig. 8 and Fig. 9 show our observation.

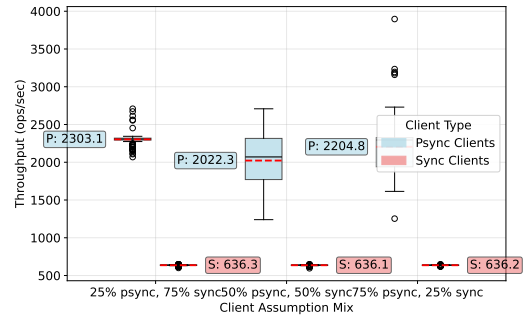


Figure 8. A box-plot showing the throughput of clients in mixed configurations. The synchronous clients perform more work verifying signatures (have poorer throughput) while enjoying improved fault-tolerance, while the partially synchronous clients enjoy better throughput, on average +242.1% with lower fault-tolerance.

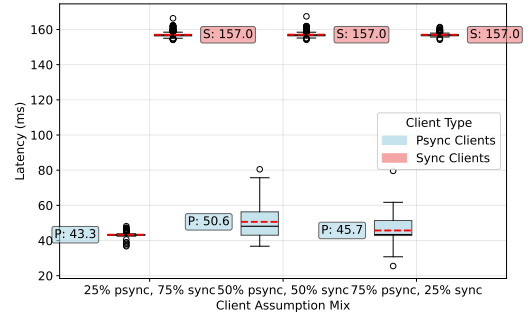


Figure 9. A box-plot showing the latency of clients in mixed configurations. The synchronous clients perform more work verifying the signatures and have poorer latencies while enjoying improved fault-tolerance, while the partially synchronous clients enjoy better latencies, on average +70.4% with lower fault-tolerance.

Analysis. Our mixed assumption experiments reveal that partially synchronous clients achieve substantial performance advantages over synchronous clients, with +242.1% higher throughput and +70.4% better latency on average. This performance gap stems from the adaptive timing model of partially synchronous clients, which eliminates the fixed

49ms waiting periods required by bounded synchrony assumptions.

Importantly, these performance benefits remain consistent across all client mixing ratios, demonstrating that our system can efficiently support heterogeneous deployments where clients choose assumptions based on their individual performance and fault tolerance requirements.

Both versions directly support our core contributions of offering flexibility.

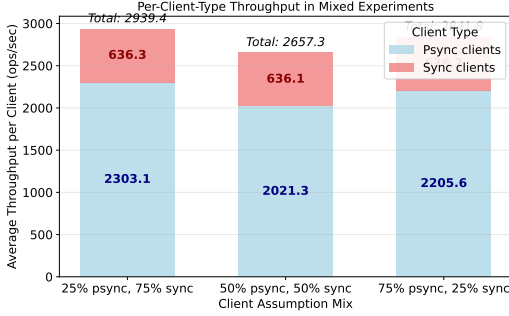


Figure 10. A plot of per-client throughput for the mixed configuration

Additionally, we explore per-client throughput in Fig. 10. We observe that the partially synchronous clients enjoy majority of the system throughput due to their adaptive nature and trade fault-tolerance for efficiency further demonstrating the costs of higher fault-tolerance.

Throughput and Latency vs. Assumption. In this section, we aim to demonstrate the impact of assumptions on throughput and latencies for the client himself.

Methodology. We run the clients with eight configurations as follows: bounded synchronous–partially synchronous, crash–Byzantine, full–no faults where full refers to the maximum faults tolerable for the chosen parameters and no faults refers to fully trusting the system with zero faults. We present our observations in Fig. 11 and Fig. 12.

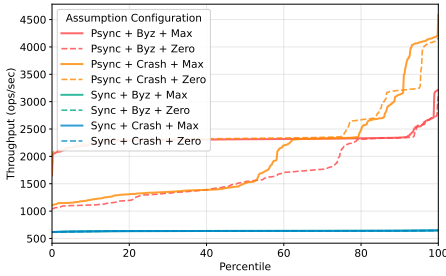


Figure 11. A histogram of the throughput for the assumption configurations.

Analysis. We observe that the partially synchronous crash none enjoys the best throughput and latencies while the bounded synchronous and Byzantine max clients have the worst throughput and latencies. This demonstrates the well known fact that trust leads to improved performance, and further motivates the need for flexible systems such as FPS. Looking at the 50th and 99th percentile numbers, trust leads to +265.5%, +535.6% improved throughput

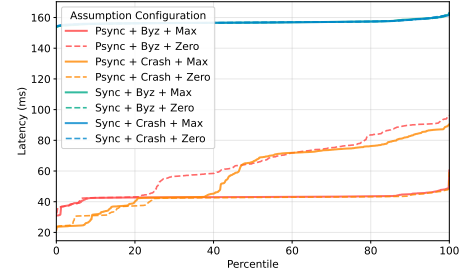


Figure 12. A histogram of the latencies for the assumption configurations.

and +72.7%, +70.6% improved latencies. These analyses underscore the need for flexible systems.

6. Related Work

We separate the related works into two sections. In the appendix, we describe works that explore consensus-free payment systems. In this part, we describe systems that explore hybrid trust.

Systems with Hybrid Trust. FlexiRaft [34] presents a flexible version of Raft [35] (a crash-fault tolerant state machine replication protocol) which supports dynamic client-side configurable quorums for data committing. Flexible BFT [11] presents a state machine replication protocol that allows the clients to choose: (i) their own values of replica faults, (ii) a bounded synchronous or partially synchronous network, and (iii) safety (correctness) and liveness (termination) or only safety. OPlex [36] presents an optimal version of FBFT where the clients can choose the same parameters but with optimal bounds as the extremes.

While these works offer flexibility for the state machine replication problem, our work offers more flexibility (more client configurable assumptions) for a payment system which is a stronger problem than the total-order implementations of the related works.

Systems with Hybrid faults. UpRight [37] presents a system that supports two types of servers: (i) servers that are *up*, i.e., always send messages when they are required to, and (ii) servers that are *right*, i.e., always send the correct messages. Using these two types of servers they show a construction of a crash-fault tolerant state machine replication with the constraint that $n > 2u + r$ where r is the number of failures that violate safety and u is the number of omission faulty servers. Visigoth-FT [38] presents a Byzantine fault-tolerant state machine replication protocol where the system consists of s slow servers, r Byzantine faults and o correlated faults. By defining u as the total number of faults (Byzantine and crash), they show that a state machine replication system requires $n > u + \min(u, s) + o$ servers which corresponds to all existing bounds for state machine replication. MixSync [39] presents a synchronous hybrid state machine replication system with $n > 2f + k$ where f is the number of Byzantine faults and k is the number of crash faults. However, all of these hybrid-fault protocols do not allow the client to choose the assumptions on the system.

7. Conclusion

We presented the first *flexible* payment system without global ordering that can tolerate clients that can make independent assumptions in three dimensions: (i) network: a choice between synchronous and partial synchronous networks with their own timeout values, (ii) faults: a choice between crash and Byzantine faults with their own number of such faults, (iii) corruption: a choice between static or adaptive corruption with their own choice of security parameter. Our implementation also demonstrated the practicality of our system.

The existence of other distributed system problems such as state machine replication and Verifiable Secret Sharing, and Multi-Party Computation protocols in the flexible model remains an interesting open problem for future research.

Disclaimer

Case studies, comparisons, statistics, research and recommendations are provided “AS IS” and intended for informational purposes only and should not be relied upon for operational, marketing, legal, technical, tax, financial or other advice. Visa Inc. neither makes any warranty or representation as to the completeness or accuracy of the information within this document, nor assumes any liability or responsibility that may result from reliance on such information. The Information contained herein is not intended as investment or legal advice, and readers are encouraged to seek the advice of a competent professional where such advice is required.

These materials and best practice recommendations are provided for informational purposes only and should not be relied upon for marketing, legal, regulatory or other advice. Recommended marketing materials should be independently evaluated in light of your specific business needs and any applicable laws and regulations. Visa is not responsible for your use of the marketing materials, best practice recommendations, or other information, including errors of any kind, contained in this document.

Ethics Considerations

This work presents a new flexible payment system design and does not identify vulnerabilities in existing deployed systems. The protocol is described at a theoretical level with implementation and evaluation conducted in a controlled research environment. This research does not involve human subjects, and the evaluation is based on simulations and performance benchmarks conducted on controlled infrastructure. No personal identifiable information (PII) or sensitive data was collected or analyzed.

References

- [1] J. S. Søgaaard, P. W. Eklund, L. Herskind, and J. Spasovski, “Improving Trust in a (Trans)National Invoicing System: The Performance of Crash vs. Byzantine Fault Tolerance at Scale,” *Applied Sciences*, vol. 13, no. 12, p. 6941, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/12/6941>
- [2] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” *Decentralized Business Review*, p. 21260, 2008.
- [3] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum Project Yellow Paper*, pp. 1–32, 2014.
- [4] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, Jul. 1982. [Online]. Available: <https://dl.acm.org/doi/10.1145/357172.357176>
- [5] J. A. Garay, A. Kiayias, and N. Leonardos, “The Bitcoin Backbone Protocol with Chains of Variable Difficulty,” 2016. [Online]. Available: <https://eprint.iacr.org/2016/1048>
- [6] D. Guo and L. Ren, “Bitcoin’s Latency–Security Analysis Made Simple,” *arXiv preprint arXiv:2203.06357*, 2022.
- [7] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D.-A. Seredinschi, “The Consensus Number of a Cryptocurrency (Extended Version),” in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, Jul. 2019, pp. 307–316. [Online]. Available: <http://arxiv.org/abs/1906.05574>
- [8] J. Sliwinski and R. Wattenhofer, “ABC: Proof-of-Stake without Consensus,” Jul. 2020. [Online]. Available: <http://arxiv.org/abs/1909.10926>
- [9] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” Nov. 2020. [Online]. Available: <http://arxiv.org/abs/2003.11506>
- [10] C. Cachin, R. Guerraoui, and L. Rodrigues, *Introduction to Reliable and Secure Distributed Programming*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011. [Online]. Available: <http://link.springer.com/10.1007/978-3-642-15260-3>
- [11] D. Malkhi, K. Nayak, and L. Ren, “Flexible Byzantine Fault Tolerance,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19. New York, NY, USA: Association for Computing Machinery, Nov. 2019, pp. 1041–1053. [Online]. Available: <https://doi.org/10.1145/3319535.3354225>
- [12] S. Gupta, “A Non-Consensus Based Decentralized Financial Transaction Processing Model with Support for Efficient Auditing - ProQuest,” [Online]. Available: <https://www.proquest.com/openview/a7fabdb0ac1f37ee5824aa89f30b904e/1?pq-origsite=gscholar&cbl=18750>
- [13] N. Maggio, “Analysing FastPay: Functionality and Formalization of FastPay,” Bachelor, University of Bern, Bern, Switzerland, Dec. 2023. [Online]. Available: <https://crypto.unibe.ch/archive/theses/2023.bsc.noah.maggio.pdf>
- [14] D. Dolev, J. Y. Halpern, B. Simons, and R. Strong, “Dynamic fault-tolerant clock synchronization,” *Journal of the ACM*, vol. 42, no. 1, pp. 143–185, Jan. 1995. [Online]. Available: <https://dl.acm.org/doi/10.1145/200836.200870>
- [15] I. Abraham, S. Devadas, D. Dolev, K. Nayak, and L. Ren, “Synchronous Byzantine Agreement with Expected $O(1)$ Rounds, Expected $\$O(n^2)\$$ Communication, and Optimal Resilience,” in *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Goldberg Ian and T. and Moore, Eds. Cham: Springer International Publishing, 2019, vol. 11598 LNCS, pp. 320–334. [Online]. Available: http://link.springer.com/10.1007/978-3-030-32101-7_20
- [16] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, Apr. 1988. [Online]. Available: <https://dl.acm.org/doi/10.1145/42282.42283>
- [17] B. Y. Chan and E. Shi, “Streamlet: Textbook Streamlined Blockchains,” in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, ser. AFT ’20. New York, NY, USA: Association for Computing Machinery, Oct. 2020, pp. 1–11. [Online]. Available: <https://doi.org/10.1145/3419614.3423256>

- [18] F. B. Schneider, “Implementing fault-tolerant services using the state machine approach: A tutorial,” *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, Dec. 1990. [Online]. Available: <https://dl.acm.org/doi/10.1145/98163.98167>
- [19] M. Castro and B. Liskov, “Practical byzantine fault tolerance and proactive recovery,” *ACM Transactions on Computer Systems*, vol. 20, no. 4, pp. 398–461, Nov. 2002. [Online]. Available: <https://doi.org/10.1145/571637.571640>
- [20] N. Shavit and M. Herlihy, *The Art of Multiprocessor Programming*. Elsevier, 2021. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/B9780124159501000021>
- [21] M. P. Herlihy and J. M. Wing, “Linearizability: A correctness condition for concurrent objects,” *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, Jul. 1990. [Online]. Available: <https://dl.acm.org/doi/10.1145/78969.78972>
- [22] P. Viotti and M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” Apr. 2016. [Online]. Available: <http://arxiv.org/abs/1512.00168>
- [23] Lamport, “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs,” *IEEE Transactions on Computers*, vol. C-28, no. 9, pp. 690–691, Sep. 1979. [Online]. Available: <https://ieeexplore.ieee.org/document/1675439>
- [24] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, “Secure and Efficient Asynchronous Broadcast Protocols,” in *Advances in Cryptology — CRYPTO 2001*, ser. Lecture Notes in Computer Science, J. Kilian, Ed. Berlin, Heidelberg: Springer, 2001, pp. 524–541.
- [25] A. Bhat, A. Bandrupalli, S. Bagchi, A. Kate, and M. K. Reiter, “The Unique Chain Rule and Its Applications,” in *Financial Cryptography and Data Security*, ser. Lecture Notes in Computer Science, F. Baldimtsi and C. Cachin, Eds. Cham: Springer Nature Switzerland, 2024, pp. 38–55.
- [26] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, “Algorand: Scaling Byzantine Agreements for Cryptocurrencies,” in *SOSP 2017 - Proceedings of the 26th ACM Symposium on Operating Systems Principles*. New York: ACM, 2017, pp. 51–68.
- [27] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” Internet Engineering Task Force, Request for Comments RFC 8032, Jan. 2017. [Online]. Available: <https://datatracker.ietf.org/doc/rfc8032>
- [28] A. Tómescu, A. Bhat, B. Applebaum, I. Abraham, G. Gueta, B. Pinkas, and A. Yanai, “UTT: Decentralized Ecash with Accountable Privacy,” 2022. [Online]. Available: <https://eprint.iacr.org/2022/452>
- [29] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” vol. 1, no. 1, Jan. 2022. [Online]. Available: <http://arxiv.org/abs/2201.05671>
- [30] D. Collins, R. Guerraoui, J. Komatovic, M. Monti, A. Xygkis, M. Pavlovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, “Online Payments by Merely Broadcasting Messages (Extended Version),” Apr. 2020. [Online]. Available: <http://arxiv.org/abs/2004.13184>
- [31] G. Avariikioti, E. K. Kogias, R. Wattenhofer, and D. Zindros, “Brick: Asynchronous Payment Channels,” Jun. 2020. [Online]. Available: <http://arxiv.org/abs/1905.11360>
- [32] J. Sliwinski and R. Wattenhofer, “Asynchronous Proof-of-Stake,” in *Stabilization, Safety, and Security of Distributed Systems*, C. Johnen, E. M. Schiller, and S. Schmid, Eds. Cham: Springer International Publishing, 2021, vol. 13046, pp. 194–208. [Online]. Available: https://link.springer.com/10.1007/978-3-030-91081-5_13
- [33] T. Albouy, E. Anceaume, D. Frey, M. Gustin, A. Rauch, M. Raynal, and F. Taïani, “Asynchronous BFT Asset Transfer: Quasi-Anonymous, Light, and Consensus-Free,” May 2024. [Online]. Available: <http://arxiv.org/abs/2405.18072>
- [34] R. Yadav and A. Rahut, “FlexiRaft: Flexible Quorums with Raft.”
- [35] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm.”
- [36] J. Neu, S. Sridhar, L. Yang, and D. Tse, “Optimal Flexible Consensus and its Application to Ethereum,” Aug. 2023. [Online]. Available: <http://arxiv.org/abs/2308.05096>
- [37] A. Clement, M. Kapritsos, S. Lee, Y. Wang, L. Alvisi, M. Dahlin, and T. Riche, “Upright cluster services,” in *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, ser. SOSP ’09. New York, NY, USA: Association for Computing Machinery, Oct. 2009, pp. 277–290. [Online]. Available: <https://doi.org/10.1145/1629575.1629602>
- [38] D. Porto, J. Leitão, C. Li, A. Clement, A. Kate, F. Junqueira, and R. Rodrigues, “Visigoth fault tolerance,” in *Proceedings of the Tenth European Conference on Computer Systems*, 2015, pp. 1–14.
- [39] I. Abraham, D. Dolev, A. Kagan, and G. Stern, “Authenticated Consensus in Synchronous Systems with Mixed Faults,” *Cryptology ePrint Archive*, 2022. [Online]. Available: <https://eprint.iacr.org/2022/805>

Appendix A. Closed forms for Quorums

Lemma A.1 (Sterling bound). *For any $t, n \in \mathbb{N}$ with $t \leq n$,*

$$\left(\frac{n}{t}\right)^t \leq \binom{n}{t} \leq \left(\frac{e \cdot n}{t}\right)^t$$

Lemma A.2 (Chernoff Bound). *Suppose $X_1, \dots, X_n$ are independent Bernoulli random variables. Let $X = \sum_{i=1}^n X_i$ and let $\mu = \mathbb{E}[X]$. Then,*

$$\Pr[X \geq (1 + \beta)\mu] \leq e^{-\frac{\beta^2 \mu}{2 + \beta}}$$

for any $\beta \geq 0$.

Lemma A.3 (One Honest Random Quorum). *Given a set of n nodes, t of which are faulty nodes, and a statistical security parameter κ , we can obtain a quorum of size ℓ containing at least one non-faulty node with probability $1 - \text{negl}(\kappa)$ if $e \cdot t < n$, for*

$$\ell \geq \frac{\kappa}{\log_2 \left(\frac{n}{e \cdot t}\right)}$$

Proof. The probability that all nodes are faulty in the quorum is

$$\frac{\binom{t}{\ell}}{\binom{n}{\ell}} \leq \frac{\left(\frac{e \cdot t}{\ell}\right)^\ell}{\left(\frac{n}{\ell}\right)^\ell} = \left(\frac{e \cdot t}{n}\right)^\ell$$

from the Sterling bound (Theorem A.1). If $\ell \geq \frac{\kappa}{\log_2 \left(\frac{n}{e \cdot t}\right)}$, then

$$\left(\frac{e \cdot t}{n}\right)^\ell \leq 2^{-\kappa} = \text{negl}(\kappa)$$

□

Lemma A.4 (Constant Fraction Honest Random Quorum). *Given a set of n nodes, t of which are faulty nodes, and a statistical security parameter κ , we can obtain a quorum of size ℓ containing at least $c \cdot \ell$ non-faulty nodes for $c \in (0, 1)$ and $(1 - c)n > t$ with probability $1 - \text{negl}(\kappa)$ for*

$$\ell \geq \frac{\kappa \cdot \log_e 2 \cdot n \cdot ((1 - c)n + t)}{((1 - c)n - t)^2}$$

Proof. Let $X_1, \dots, X_\ell$ denote indicator random variables corresponding to whether the i th node in the quorum is faulty or not. We have $\Pr[X_i = 1] = \frac{t}{n}$ for all $1 \leq i \leq \ell$. Let $X = \sum_{i=1}^{\ell} X_i$. We have $\mathbb{E}[X] = \ell \cdot \frac{t}{n}$. From the Chernoff bound (Theorem A.2), we have

$$\Pr \left[X \geq (1 + \beta) \frac{\ell t}{n} \right] \leq e^{-\frac{\beta^2 \ell t}{3n}} \leq 2^{-\kappa} = \text{negl}(\kappa)$$

for

$$\ell \geq \frac{(2 + \beta)\kappa \cdot \log_e 2 \cdot n}{\beta^2 t}$$

Setting $(1 + \beta) \frac{\ell t}{n} = (1 - c)\ell$, i.e., $\beta = (1 - c) \frac{n}{t} - 1 = \frac{(1-c)n-t}{t}$ gives the bound in the lemma. Finally, note that if $X < (1 - c)\ell$, then the number of non-faulty nodes is more than $c \cdot \ell$, as required. $\square$

Appendix B. Discussion

Cross assumption payments. Payments between clients with different (potentially conflicting) assumptions are not guaranteed security in our protocol. We assume that trusted entities such as gateways will exist in the system to facilitate payments across assumptions. These gateways may charge deposits or fees proportional to the risk they undertake. We also find this an interesting problem in general as it is unproven how to make systems with different work together. For instance, consider bridge or swap protocols across two blockchains with different assumptions. They encounter a similar problem and solving it is an interesting but orthogonal future work.

Reset Assumptions. Our system is secure as long as a client makes one assumption and does not change its assumption. However, a practical system requires that clients can update this assumption. We can extend FPS, with an out-of-band mechanism where the faults are *repaired* every epoch, say every day. Since monetary systems are highly regulated, we can assume that the regulator resolves any disputes and thus ensures that all servers have the same state. This allows the clients to change their assumption every epoch.

Flexible BFT. We can view our system as a Layer-2 scalability solution to an existing Layer 1 (total-order based) system. A key problem here is that the assumptions of the Layer 1 solution (for example, Bitcoin requires 1/3 Byzantine faults and a 10 minute bounded synchronous network) might conflict with the assumptions of the clients thereby rendering most assumptions of FPS useless. An interesting future work here is to design and develop a similar flexible Layer 1 solution. We can now use FPS as a Layer 2 solution. We find our work as an exciting start to new future work in this direction.

Anonymity. We can add anonymity to our solution by using cryptographic techniques from UTT [28] or Zef [29] to define a payment instruction that proves that the sender has sufficient balance, the sender has an authorization to pay, and that the sender is not double spending.

Concurrent Payments. Our system can allow concurrent payments by relaxing the restriction that a non-faulty client is sequential to allowing them to send a batch of payments with continuous sequence numbers and/or allow the payment transaction to include multiple receivers and multiple amounts with the same sequence number.

Appendix C. Extended Related Works

Consensus-free payment systems. Saurabh Gupta [12] was the first to propose a Byzantine fault-tolerant permissioned asynchronous payment system that does not rely on consensus. Guerraoui et al. [7] present a Byzantine fault-tolerant asynchronous payment system using secure broadcast with source order as its primitive. They present the system as an implementation of the asset transfer abstraction in the Byzantine message passing model. Astro [30], Brick [31], and FastPay [9] are concurrent works. Astro [30] presents an Byzantine fault-tolerant asynchronous payment system where each client has its own *exclusive log*, using Byzantine Reliable Broadcast as its primitive. Brick [31] presents a payment system as a payment channel to blockchain protocols. They use an external set of *wardens* and clients use consistent broadcast between the wardens to perform payments. FastPay [9] presents an asynchronous Byzantine fault-tolerant payment system with an implementation using Byzantine consistent broadcast. ABC [8] and Cascade [32] presented an asynchronous blockchain using a redefined version of consensus with termination only guaranteed for non-faulty clients. UTT [28] and Zef [29] are concurrent works that present Byzantine fault-tolerant privacy preserving payment systems. UTT presents an auditable partially synchronous privacy-preserving payment system and presents a consensus-free fast-path using Byzantine consistent broadcast. Zef [29] presents a privacy-preserving asynchronous Byzantine fault-tolerant payment system. Noah Maggio [13] presents a detailed analysis of FastPay [9] and re-classifies the FastPay protocol as an implementation of a payment system using hybrid Byzantine Consistent Broadcast. Finally, Albouy et al. [33] present a privacy preserving payment system as an implementation of the asset transfer problem in the Byzantine message passing model.

Appendix D. Security Analysis

Proof of generic compilation. We prove the security of the generic construction of a payment system using an HCB-SO instance. The key challenge is to prove the correctness of the recursive dependency of payments and we use induction in Theorem D.1 to prove it.

Lemma D.1 (Consistent partial ordering). *In Fig. 2, if a transfer operation Transfer is successfully executed by a non-faulty node, then all non-faulty nodes successfully execute Transfer.*

Proof. We will prove this using strong induction on the global dependency graph of an account.

For each account A , let $T_1, T_2, \dots$, denote the sequence of outgoing transfers from A in the order they are broadcast. Define the dependency set of transfer T_i as

$$\text{deps}(T_i) = \text{lastIn} \cup \text{deps}(T_{i-1})$$

where lastIn is the queue maintained by the client owned by the sending account and $\text{deps}(T_1) := \text{lastIn} = \emptyset$.

We define the *depth* of a transfer T_i as

$$\text{depth}(T_i) = |\text{deps}(T_i)|$$

Base Case. For the first transfer operation T_1 from any account A successfully executed by non-faulty node n_i , the dependencies in lastIn are $\emptyset$ (Line 7), thus $\text{depth}(T_1) = 0$.

Consider any non-faulty node n_j :

- From HBCB Consistency (Definition 3.2): node n_j delivers the same message T_1 with the same $(A, A', v, \emptyset)$.
- Validation (Lines 17–18): authentication and balance checks (Line 25) pass, since $\emptyset \subseteq \text{hist}[A]$ holds trivially, and T_1 is the first transfer from A , we have $\text{hist}[A] = \emptyset$ at all nodes. Thus, $\text{Balance}(A, \text{hist}[A])$ returns the initial balance of A at all nodes. Since the check passed at node n_i , the initial balance satisfies $A.\text{bal} \geq v$, hence all nodes have the same initial balance, so the check passes at node n_j .

Therefore, all non-faulty nodes successfully execute T_1 .

Induction Hypothesis. For all transfers T with $\text{depth}(T) < k$: if a non-faulty node successfully executes T , then all non-faulty nodes successfully execute T .

Induction Step. Let T_i be a transfer from account A with $\text{depth}(T_i) = k > 0$, successfully executed by non-faulty node n_i .

Since node n_i successfully executed T_i :

1) Node n_i validated that $\text{lastIn} \subseteq \text{hist}[A]$ (Line 18), meaning all transfers in lastIn were successfully executed at node n_i .

2) We have $\text{deps}(T_i) = \text{lastIn} \cup \text{deps}(T_{i-1})$. For any $T' \in \text{deps}(T_i)$, we have $\text{deps}(T') \subset \text{deps}(T_i)$, thus $\text{depth}(T') < k$.

3) By the induction hypothesis: all non-faulty nodes successfully executed each $T' \in \text{deps}(T_i)$.

4) From HBCB Consistency (Definition 3.2): all non-faulty nodes deliver the same message T_i with the same $(A, A', v, \text{lastIn})$.

5) From HBCB Source Order (Definition 3.2): all non-faulty nodes deliver transfers $T_1, \dots, T_i$ from A in the same order.

6) Consider any non-faulty node n_j :

- By the induction hypothesis, all $T' \in \text{lastIn}$ were successfully executed at node n_j , thus $\text{lastIn} \subseteq \text{hist}[A]$ and validation (Line 18) passes.
- By the induction hypothesis and Source Order, all transfers in $\text{deps}(T_i)$ were successfully executed at node n_j in the same order as at node n_i .
- Therefore, $\text{hist}[A]$ contains the same set of successfully executed transfers at both nodes, and $\text{Balance}(A, \text{hist}[A])$ computes the same value.

- Since the balance check (Line 25) passed at node n_i , it passes at node n_j .

7) Node n_j successfully executes T_i (Lines 50–51).

By strong induction, all non-faulty nodes successfully execute every transfer that is successfully executed by any non-faulty node. $\square$

We now present the formal guarantees in the following theorem:

Theorem D.2. *Fig. 2 implements a payment system (from Definition 3.1).*

Proof. Let E be any execution of an implementation and H be the corresponding history. Let $\text{ops}(H)$ denote the set of operations in history H executed by non-faulty clients. We split the proof into three parts.

Proof of Termination. Termination for Balance holds since it is a local operation (Line 2). For Transfer: from Theorem D.1, if a non-faulty client broadcasts a transfer, all non-faulty clients eventually execute it (successfully or as failed). Combined with the termination of the local Balance check, Transfer terminates.

Proof of Correctness. Let $\bar{H}$ be a completion of H obtained by removing incomplete invocations. Let $\bar{H}^T$ denote the sub-history of successful transfers of $\bar{H}$ performed by non-faulty nodes. Let $\prec_{\bar{H}}$ be the real-time ordering over $\bar{H}$, and let $\prec_{\bar{H}}^T$ denote the restriction of $\prec_{\bar{H}}$ to operations in $\bar{H}^T$.

We construct a partial order $\prec$ over successful transfers as follows:

- For transfers from the same account A : if T_i^A and T_j^A with $i < j$, then $T_i^A \prec T_j^A$ (from HBCB Source Order).
- For transfers with explicit dependencies: if $T' \in \text{lastIn}_T$, then $T' \prec T$ (from Line 18).
- Take the transitive closure of these relations.

From Theorem D.1: all non-faulty nodes successfully execute the same set of transfers. Thus, the partial order $\prec$ is consistent across all non-faulty nodes.

By Szpilrajn's extension theorem, we can extend the partial order $\prec$ to a total order $\prec_S$ that preserves $\prec$. Let S be a sequential history corresponding to any such extension.

We show S satisfies linearizability:

1) For every non-faulty node n_i : From Theorem D.1, node n_i successfully executes exactly the transfers that appear in S . Thus, $\bar{H}^T|_{n_i} = S|_{n_i}$.

2) Real-time ordering is preserved: If transfer T completes before T' starts in $\bar{H}$ (i.e., $T \prec_{\bar{H}} T'$), then either:

- Both are from the same account: Source Order ensures $T \prec T'$, thus $T \prec_S T'$.
- T is an incoming transfer to the sender of T' : Then $T \in \text{lastIn}_{T'}$, so $T \prec T'$, thus $T \prec_S T'$.
- Otherwise: the orderings are concurrent, and any extension is valid.

Thus, $\prec_{\bar{H}}^T \subseteq \prec_S$.

Proof of Soundness. We show that for each non-faulty node n_i , there exists a sequential history S_i satisfying sequential consistency.

Successful transfers: From the correctness proof, there exists S such that $\bar{H}^T|_{n_i} = S|_{n_i}$.

Balance operations: These are local (Line 2) and always succeed at non-faulty nodes.

Failed transfers at node n_i : These fail due to either (i) invalid authentication (Line 17), or (ii) insufficient balance (Line 25).

- Case (i): Authentication failures are determined locally and are consistent with any sequential history.
- Case (ii): The failure is based on $\text{Balance}(A, \text{hist}[A])$ which depends only on successfully executed transfers in $\text{hist}[A]$. Since these are linearizable (from correctness), the failure reason is consistent with S .

Construct S_i by interleaving the sequential history S for successful transfers with balance operations and failed transfers observed by node n_i , respecting the local ordering at node n_i . This gives a legal sequential history satisfying:

- $\text{ops}(\bar{H}) \subseteq \text{ops}(S_i)$ (all operations appear)
- $S_i|_{n_i} = \bar{H}|_{n_i}$ (respects local view)

□

Security of FPS. We show that Fig. 2 implements a payment system (Definition 3.1) when given access to a primitive *hbc-b-so* that satisfies the properties of hybrid Consistent Broadcast Definition 3.2.

Theorem D.3. *For a $N_S C_* F_C$ -client, for any $t < n$, the FPS system implements a secure hybrid BCB with SO.*

Proof. We prove safety and liveness separately.

Safety. Suppose a non-faulty recipient successfully finalizes a payment pay with sequence number sn from sender A_s . The recipient obtained a confirmation quorum certificate from $t+1$ servers and waited $2\Delta_C + \Delta_S$ without receiving equivocation errors. During this timeout, all non-faulty servers (at least one among the $t+1$ contacted) multicast pay to all servers. By time $2\Delta_C + \Delta_S$, all $n-t$ non-faulty servers receive and process pay , marking $\text{status}[A_s, sn] = \text{Confirmed}(pay)$.

If a Byzantine sender attempts to broadcast a conflicting payment $pay' \neq pay$ with the same sn , any non-faulty server that already processed pay detects equivocation and sends an error to the recipient. Since the recipient waits $2\Delta_C + \Delta_S$ and hears no errors, no conflicting payment exists at any non-faulty server. During finalization, the recipient sends to $t+1$ servers; at least one is non-faulty and has $\text{status}[A_s, sn] = \text{Confirmed}(pay)$, ensuring consistency. Thus, only one payment per sequence number can be finalized.

Liveness. With $t < n$, we have $n-t \geq 1$ non-faulty servers. When the recipient multicasts to $t+1$ servers, at least one is non-faulty (since $t+1 > t$). This non-faulty server multicasts pay to all servers, ensuring all $n-t$ non-faulty servers receive pay within Δ_S . The recipient waits $2\Delta_C + \Delta_S$, collects at least one confirmation, and proceeds to finalization. Finalization similarly succeeds with at least one non-faulty server among the $t+1$ contacted. Thus, $t < n$ is necessary and sufficient for liveness. □

Theorem D.4. *For a $N_S C_* F_B$ -client, for any $t < n/2$, the FPS system implements a secure hybrid BCB with SO.*

Proof. We prove safety and liveness separately.

Safety. Suppose a non-faulty recipient successfully finalizes a payment pay with sequence number sn from sender A_s . The recipient obtained a confirmation quorum certificate from $t+1$ servers and waited $2\Delta_C + \Delta_S$ without receiving equivocation errors. During this timeout, all non-faulty servers (at least one among the $t+1$ contacted) multicast pay to all servers. By time $2\Delta_C + \Delta_S$, all $n-t$ non-faulty servers receive and process pay , marking $\text{status}[A_s, sn] = \text{Confirmed}(pay)$.

If a Byzantine sender attempts to broadcast a conflicting payment $pay' \neq pay$ with the same sn , any non-faulty server that already processed pay detects equivocation and sends an error to the recipient. Since the recipient waits $2\Delta_C + \Delta_S$ and hears no errors, no conflicting payment exists at any non-faulty server. During finalization, the recipient sends to $2t+1$ servers. With $t < n/2$ (thus $n > 2t$), we have $2t+1 \leq n$, so the recipient can contact $2t+1$ servers. Among these $2t+1$ servers, at least $2t+1-t = t+1 > t$ are non-faulty, and all have $\text{status}[A_s, sn] = \text{Confirmed}(pay)$. Byzantine servers cannot produce a valid finalization certificate for pay' since they control at most t servers, which is insufficient to form a majority among the $2t+1$ contacted. Thus, only one payment per sequence number can be finalized.

Liveness. With $t < n/2$, we have $n-t > n/2$, ensuring at least $\lceil n/2 \rceil$ non-faulty servers. When the recipient multicasts to $t+1$ servers, at least one is non-faulty (since $t+1 > t$). This non-faulty server multicasts pay to all servers, ensuring all $n-t$ non-faulty servers receive pay within Δ_S . The recipient waits $2\Delta_C + \Delta_S$, collects at least one confirmation, and proceeds to finalization. For finalization, the recipient contacts $2t+1$ servers; with $n > 2t$, this is feasible. Among the $2t+1$ servers contacted, at least $t+1$ are non-faulty and respond with valid finalization acknowledgments. Thus, $t < n/2$ is necessary and sufficient for liveness. □

Theorem D.5. *For a $N_P C_* F_C$ -client, for any $t < n/2$, the FPS system implements a secure hybrid BCB with SO.*

Proof. We prove safety and liveness separately.

Safety. The protocol uses a two-phase approach: confirmation followed by finalization. For confirmation, the recipient waits for acknowledgments from a quorum of size $q_c = \lceil n/2 \rceil + 1$ servers. Suppose two conflicting payments pay and pay' (with the same sequence number sn from sender A_s) both obtain confirmation certificates.

Since crash faults do not equivocate, each non-faulty server sends at most one confirmation per sequence number. Two quorums of size q_c must overlap: $2q_c > n$ implies the intersection contains at least $2q_c - n \geq 2(\lceil n/2 \rceil + 1) - n \geq 1$ server. Since crashed servers do not respond, any responding server in the intersection is non-faulty and could not have confirmed both pay and pay' . Thus, both payments cannot obtain valid confirmation certificates.

During finalization, the recipient multicasts the certificate to $t + 1$ servers. Servers in *Locked* state only apply the payment matching the certificate. Since only one payment obtained a confirmation certificate, only that payment can be finalized.

Liveness. For liveness, we need $q_c \leq n - t$ (enough non-faulty servers to form a quorum). We have $\lceil n/2 \rceil + 1 \leq n - t$. For n even: $n/2 + 1 \leq n - t$ gives $t \leq n/2 - 1 < n/2$. For n odd: $(n + 1)/2 + 1 \leq n - t$ gives $t \leq (n - 3)/2 < n/2$. Thus, $t < n/2$ ensures liveness. With $t < n/2$, the recipient can collect q_c confirmations and finalize with $t + 1$ servers (at least one non-faulty). $\square$

Theorem D.6. *For a $N_P C_* F_B$ -client, for any $t < n/3$, the FPS system implements a secure hybrid BCB with SO.*

Proof. We prove safety and liveness separately.

Safety. The protocol uses a two-phase approach: confirmation followed by finalization. For confirmation, the recipient waits for acknowledgments from a quorum of size $q_b = \lceil (n + t)/2 \rceil + 1$ servers. Suppose two conflicting payments pay and pay' (with the same sequence number sn from sender A_s) both obtain confirmation certificates.

Byzantine servers may equivocate, sending confirmations for both pay and pay' . However, two quorums of size q_b must overlap in at least $2q_b - n$ servers. We have $2q_b - n \geq 2(\lceil (n + t)/2 \rceil + 1) - n \geq 2((n + t)/2 + 1) - n = t + 2$. Since at most t servers are Byzantine, the intersection contains at least $(t + 2) - t = 2$ non-faulty servers. However, each non-faulty server confirms at most one payment per sequence number. For both pay and pay' to obtain valid quorums, at least one non-faulty server must confirm both, which is a contradiction. Thus, at most one payment can obtain a valid confirmation certificate.

During finalization, the recipient multicasts the certificate to $t + 1$ servers. Servers in *Locked* state only apply the payment matching the certificate. Since only one payment obtained a confirmation certificate, only that payment can be finalized.

Liveness. For liveness, we need $q_b \leq n - t$ (enough non-faulty servers to form a quorum). We have $\lceil (n + t)/2 \rceil + 1 \leq n - t$. This simplifies to $(n + t)/2 + 1 \leq n - t$, giving $n + t + 2 \leq 2n - 2t$, thus $3t \leq n - 2$. For $t \geq 1$, we have $t < n/3$. Thus, $t < n/3$ ensures liveness. With $t < n/3$, the recipient can collect q_b confirmations from non-faulty servers and finalize with $t + 1$ servers (at least one non-faulty). $\square$

Theorem D.7. *For a $N_* C_S F_*$ -client, for any probabilistic quorum ensuring the ratio of faults the same as that of $N_* C_A F_*$, the FPS system implements a secure hybrid BCB with SO.*

Proof. This follows trivially from the adaptive arguments and the fact that the probability of a static adversary violating the bounds is negligible in the security parameter κ . $\square$